

PROJECT REPORT

E-Commerce Web Automation
Using Selenium WebDriver with Python

CAPSTONE PROJECT

Training Program: WIPRO Python Automation Course

Field	Details
Name	Ujjawal Kumar
Enrollment No.	12023002029113
Stream	CSE(IoT, CS & BT)
Campus	IEM Newtown
Application Under Test	AutomationExercise

1. Objective

The objective of this project is to design and implement a functional web automation solution for an e-commerce application using Python and Selenium WebDriver. The implementation automates key customer actions and validates that the application behaves as expected during a basic purchase workflow.

The project demonstrates the following core automation concepts:

- Browser launch and navigation using Selenium WebDriver.
- User login automation using test credentials stored separately from the test logic.
- Product search and navigation to the product details page.
- Adding a product to the shopping cart and handling the add-to-cart popup.
- Updating product quantity and validating the resulting cart values.
- Reading test inputs from JSON test data.
- Capturing screenshots at important execution checkpoints.
- Generating a simple execution report containing verification results.

2. Tools & Technologies

Tool / Technology	Version / Type	Purpose
Python	3.x	Core programming language used to write automation scripts.
Selenium WebDriver	4.49.0	Browser automation and element interaction.
Google Chrome	Installed browser	Browser used for test execution.
JSON	Python standard library	Stores test credentials, product information and quantity inputs.
Git / GitHub	Version control	Stores and tracks the project source code and documentation.
VS Code	Development environment	Used to develop, execute and debug the Python scripts.

3. Application Under Test

AutomationExercise is a publicly accessible e-commerce practice website used for browser automation and functional testing.

- Base URL: <https://automationexercise.com/>
- Application areas tested: user login, product search, product details, add to cart, quantity update and cart verification.
- Product used in the demonstrated workflow: Men Tshirt.
- Observed verification values for the demonstrated quantity update: quantity 4, unit price Rs. 400, total Rs. 1600.

4. Project Architecture

The Capstone 1 implementation follows a script-based structure. Each Python file represents one logical automation step so that individual actions can be executed, demonstrated and debugged independently.

```
Capstone_1_Selenium_ECommerce/
├── src/
│   ├── login_test.py
│   ├── product_search.py
│   ├── add_to_cart.py
│   ├── update_quantity.py
│   └── verify_cart.py
├── test_data/
│   └── test_data.json
├── screenshots/
│   ├── 01_login_success.png
│   ├── 02_product_search.png
│   ├── 03_product_added.png
│   ├── 04_quantity_updated.png
│   └── 05_cart_verification.png
├── outputs/
├── reports/
│   └── execution_report.txt
├── README.md
└── requirements.txt
```

The implementation intentionally avoids an unnecessary framework layer for this first capstone and instead focuses on demonstrating the requested Selenium WebDriver operations clearly.

5. Test Cases

Test ID	Test / Script	Type	Description	Expected Result
TC-01	login_test.py	Functional	Open the application and log in using the configured test credentials.	Login succeeds and the authenticated page is displayed.
TC-02	product_search.py	Functional	Navigate to Products and search for the target product.	Matching product result is displayed.
TC-03	add_to_cart.py	Functional	Open product details, set the quantity and add the product to the cart. Handle the confirmation popup.	Product is added to the cart and the cart can be opened.
TC-04	update_quantity.py	Functional	Update the cart quantity from 1 to 4.	Cart quantity changes to 4.
TC-05	verify_cart.py	Validation	Read product, price, quantity and total from the cart and compare against the expected total.	Product details and calculated total match the expected values; a report is generated.

6. Test Data Management

Test-specific values are kept in a JSON file so that the automation logic is separated from frequently changing inputs. This makes it possible to change credentials, product information or quantity without rewriting the Selenium actions.

```
{
  "username": "<test-account-email>",
  "password": "<test-account-password>",
  "product": "Men Tshirt",
  "quantity": 4,
  "expected_unit_price": 400,
  "expected_total": 1600
}
```

Credentials shown above are represented as placeholders in the report. The working project uses the local test-data file and does not publish credentials in the report.

7. Key Implementation Details

7.1 Browser Launch and Login

Selenium creates a Chrome WebDriver session and navigates to the application. The login script locates the email and password fields, enters the test credentials and submits the login form. A screenshot is captured after a successful login for evidence.

```
driver = webdriver.Chrome()
driver.get("https://automationexercise.com/")

# Locate credentials fields and submit login
email = driver.find_element(By.NAME, "email")
password = driver.find_element(By.NAME, "password")
email.send_keys(username)
password.send_keys(user_password)
```

7.2 Product Search

The product search script navigates to the Products page, enters the target product in the search field and submits the search. The resulting product listing is validated visually and saved as a screenshot.

```
search_box = driver.find_element(By.ID, "search_product")
search_box.send_keys("Men Tshirt")
driver.find_element(By.ID, "submit_search").click()
```

7.3 Add Product to Cart

After locating the target product, the automation opens its details page, sets the requested quantity and clicks the Add to Cart button. The confirmation modal is then dismissed and the cart page is opened.

```
driver.find_element(By.CSS_SELECTOR, "a[href=/product_details/2]").click()
quantity = driver.find_element(By.ID, "quantity")
quantity.clear()
quantity.send_keys(str(test_quantity))
driver.find_element(By.CSS_SELECTOR, "button.cart").click()
```

7.4 Quantity Update

The quantity is updated directly in the cart. The implemented test demonstrated changing the product quantity from 1 to 4 successfully.

7.5 Cart Verification

The verification script reads the product name, unit price, quantity and total from the cart. It calculates the expected total from unit price and quantity and compares it with the value displayed by the application.

```
expected_total = unit_price * quantity
actual_total = cart_total

if actual_total == expected_total:
    print("Cart verification passed")
```

8. Evidence & Outputs

The project stores screenshots at major execution checkpoints. These files provide evidence that each stage of the automation completed successfully.

Screenshot	Purpose
01_login_success.png	Evidence of successful login.
02_product_search.png	Evidence of the searched product and result page.
03_product_added.png	Evidence after the product is added to the cart.
04_quantity_updated.png	Evidence that the cart quantity was changed to 4.
05_cart_verification.png	Evidence of final cart values and verification.

8.1 Execution Report

A plain-text execution report is generated under the reports folder. It records the verification result and relevant cart information from the execution. The report is intentionally lightweight and easy to inspect without additional reporting dependencies.

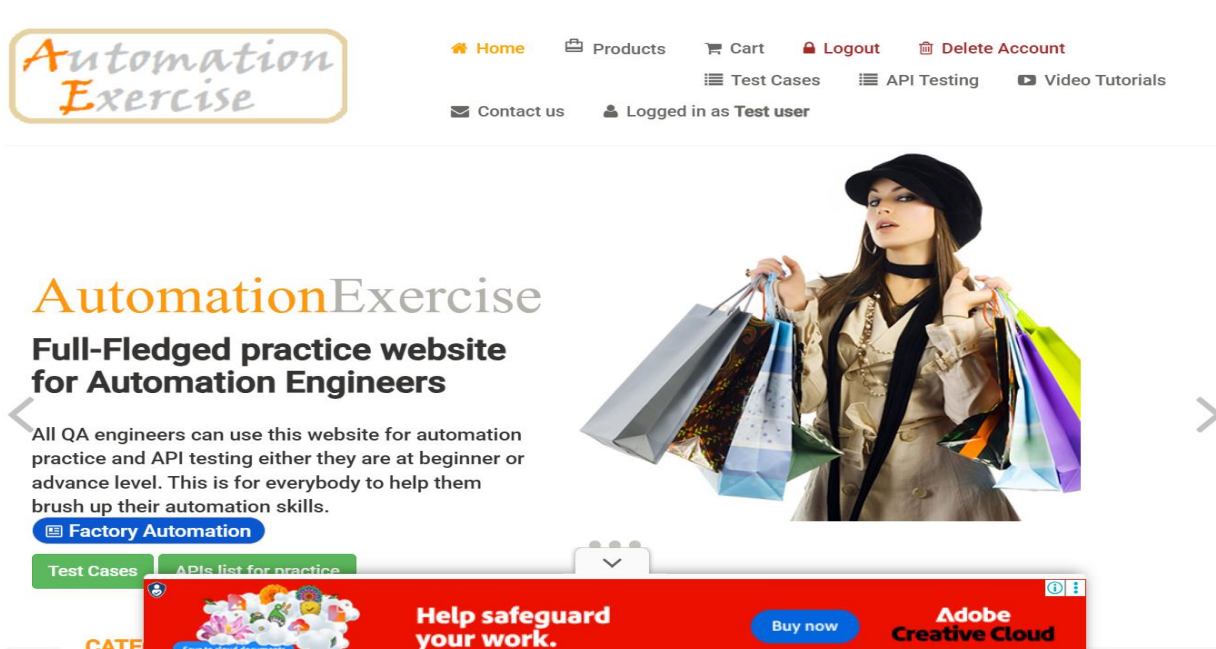


Figure 1 : 01_Login_Success.png

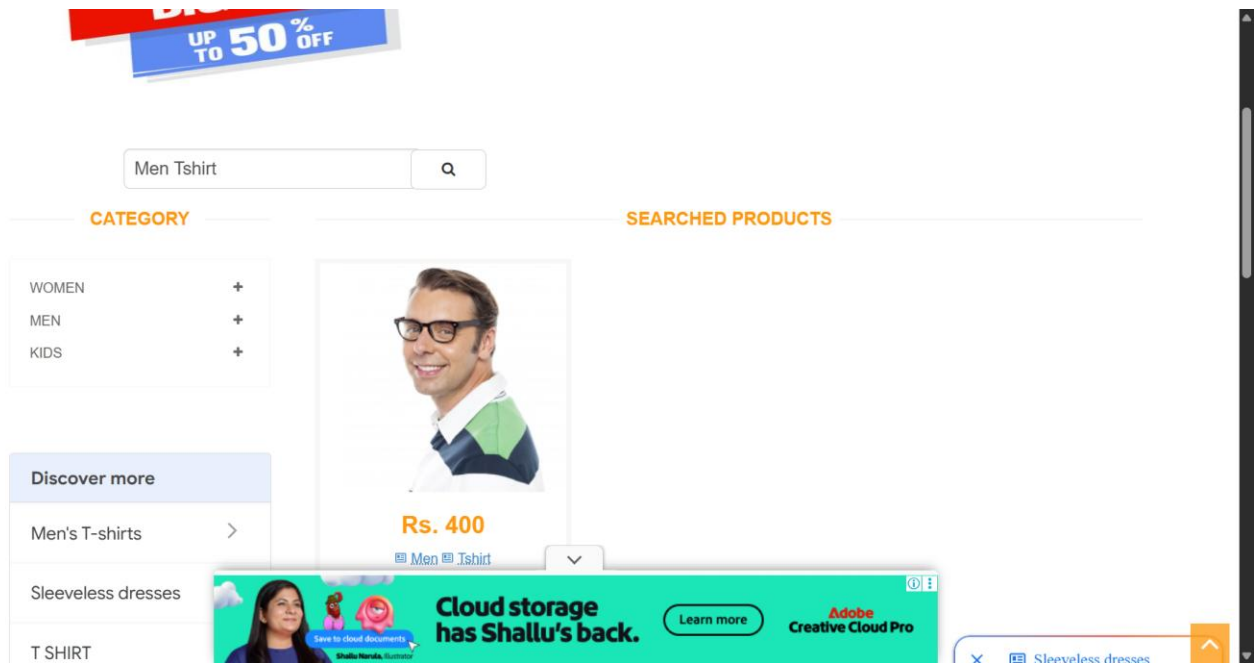


Figure 2 : 02_Product_search.png

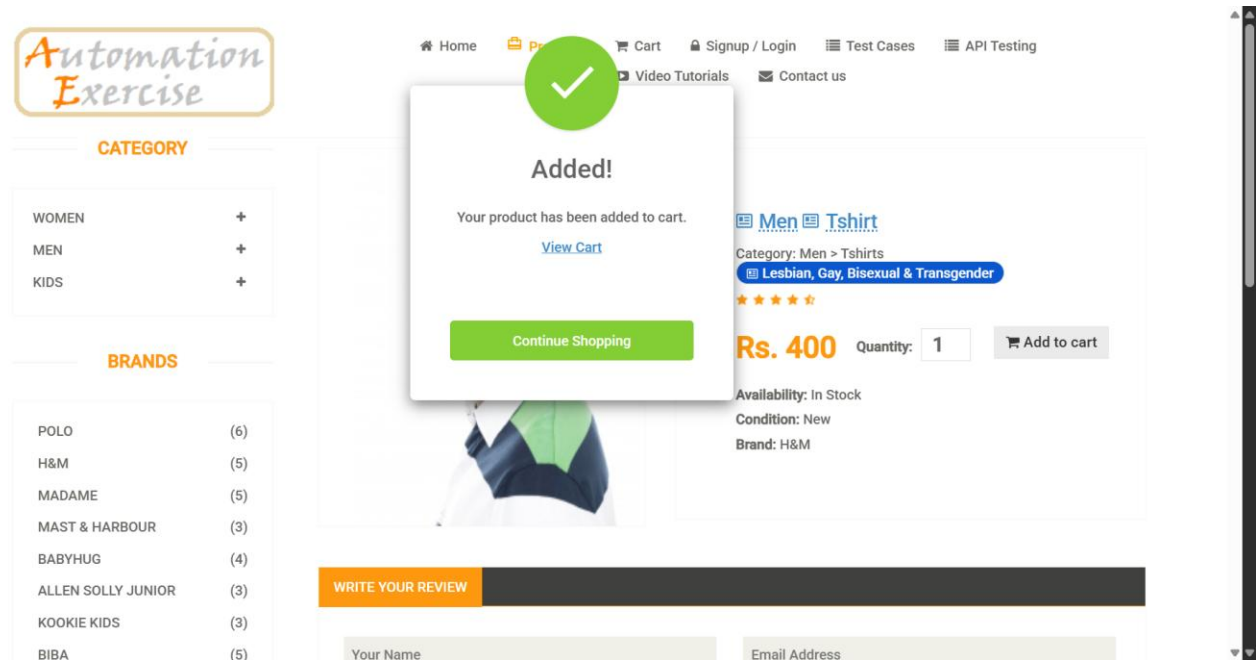


Figure 3 : 03_product_add.png

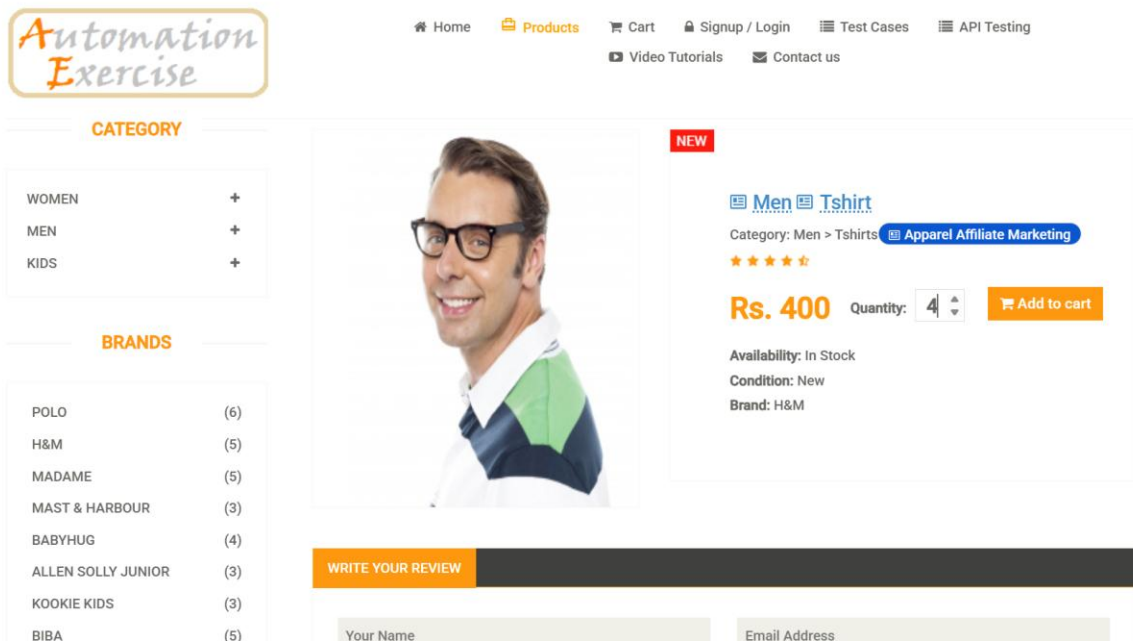


Figure 4 : 04_quantity_updated.png

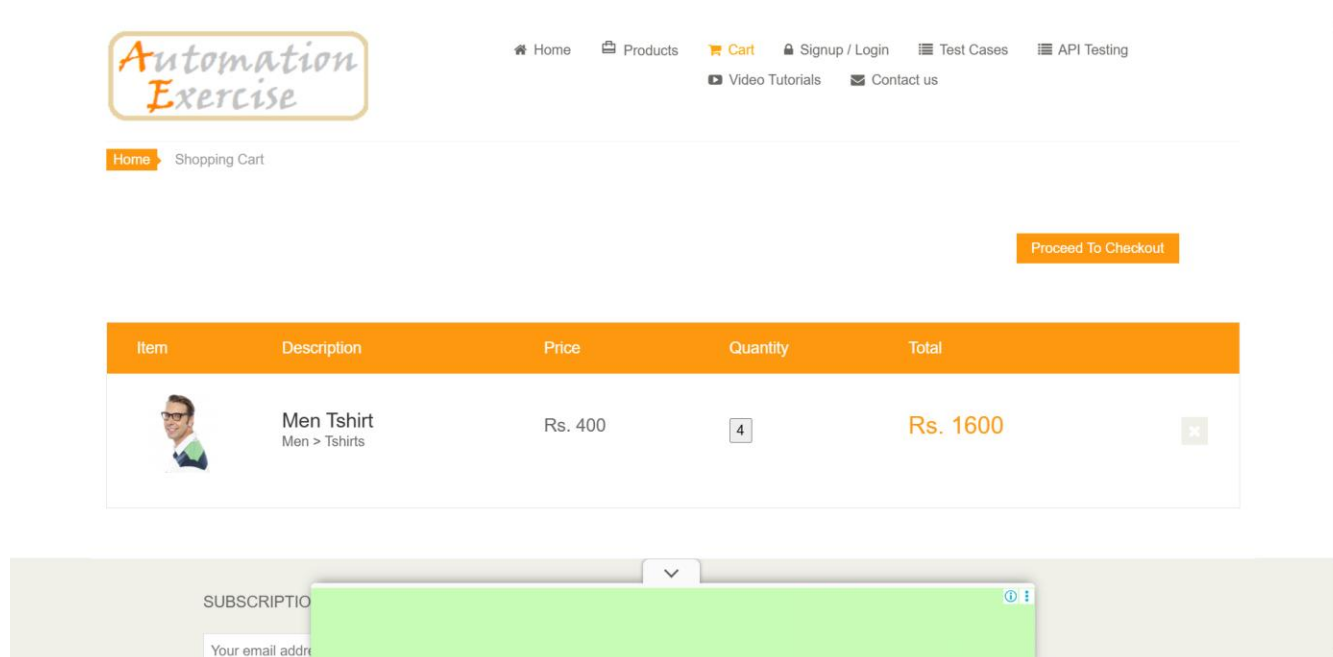


Figure 5 : 04_quantity_updated-cart.png

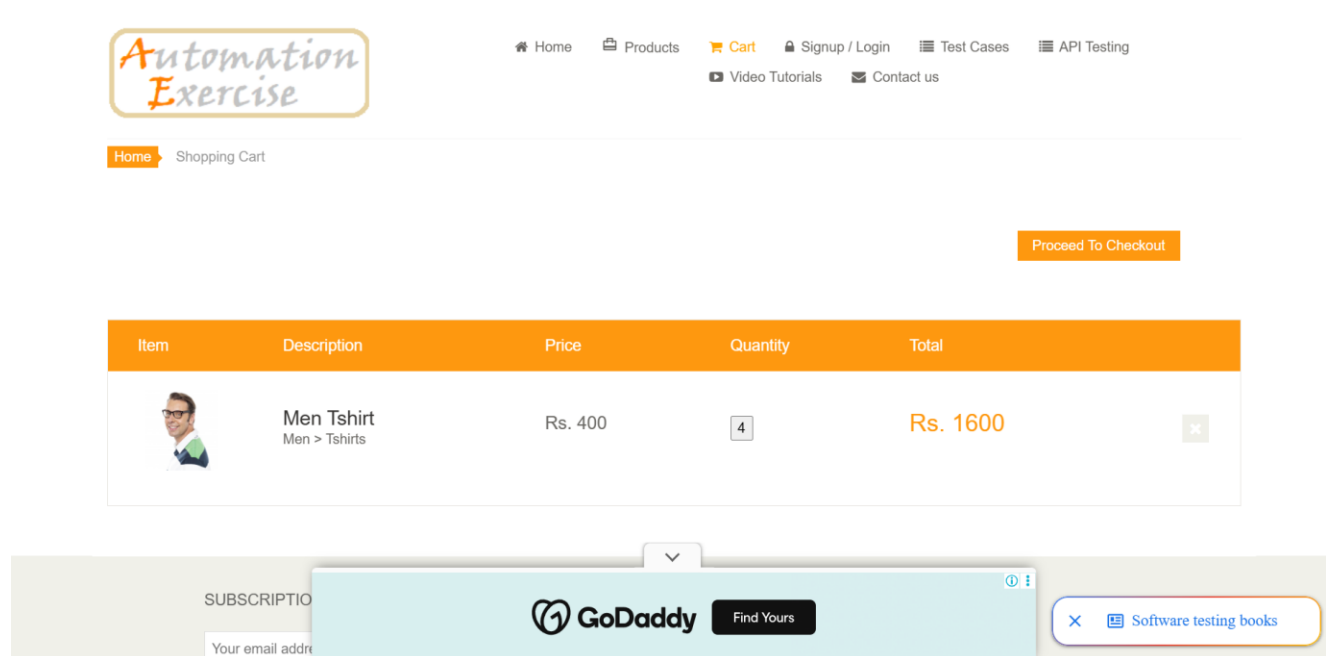


Figure 6 : 06_cart_verification.png

9. Challenges Faced & Solutions

Challenge	Solution / Learning
Products on the initially selected Tutorialsninja site were unavailable for the required workflow.	The application under test was changed to AutomationExercise, where the required product and quantity workflow could be demonstrated.
A dynamic Google advertisement iframe occasionally intercepted clicks.	The issue was investigated by inspecting the page structure; scrolling and JavaScript click techniques were considered when a normal click was intercepted.
Cart verification could be affected by transient page overlays.	The verification script was executed when the relevant cart elements were accessible and the selectors were based on the cart row structure.
Element locators had to be adapted to the actual HTML of the target application.	The browser developer tools were used to inspect IDs, CSS selectors and element structure before finalizing the locators.
Test inputs needed to be changed without editing scripts repeatedly.	Inputs such as product and quantity were placed in a JSON test-data file.

10. How to Run the Project

- Install Python 3.x and Google Chrome.
- Clone or copy the GitHub repository to the local system.
- Create and activate a Python virtual environment inside the Capstone 1 folder.
- Install dependencies using the supplied requirements.txt file.
- Create the local test_data/test_data.json file using the project format.
- Run the scripts from the src folder in logical order: login_test.py, product_search.py, add_to_cart.py, update_quantity.py and verify_cart.py.
- Review the screenshots folder and reports/execution_report.txt for execution evidence.

```
pip install -r requirements.txt
python src/login_test.py
python src/product_search.py
python src/add_to_cart.py
python src/update_quantity.py
python src/verify_cart.py
```

11. Additional Selenium Demonstration: Tab Handling

As part of the Selenium practice work accompanying the project, tab and window-handle operations were also implemented. The demonstration opens an initial URL, launches a new tab, navigates to another URL, performs an input action, closes the second tab and returns to the first tab.

```
driver.get("https://tutorialsninja.com/demo")
first_tab = driver.current_window_handle

driver.switch_to.new_window("tab")
driver.get("https://testautomationpractice.blogspot.com/")

text_box = driver.find_element(By.ID, "name")
text_box.send_keys("Ujjawal Kumar")

driver.close()
driver.switch_to.window(first_tab)
```

This exercise demonstrates window handles, tab creation, explicit switching between browser contexts and controlled tab closure. The implementation video can be used as execution evidence for this part.

12. Conclusion

The project successfully demonstrates practical browser automation using Python and Selenium WebDriver on an e-commerce application. The implementation covers the core workflow required for a customer purchase scenario: login, product search, product selection, cart addition, quantity update and cart validation. Supporting evidence is produced through screenshots and a text execution report, while test values are kept in JSON to improve maintainability.

The work also provided practical exposure to web element inspection, CSS and ID-based locators, browser tabs, popup handling, debugging of dynamic page behavior and version-controlled project organization. The resulting implementation provides a clear foundation for the more structured Selenium frameworks introduced in later capstone assignments, such as PyTest, Page Object Model and utility layers.

Appendix A. Important Selenium Locators Used

Element / Action	Locator
Products navigation	a[href="/products"]
Product search field	#search_product
Search button	#submit_search
Men Tshirt details	a[href="/product_details/2"]
Quantity field	#quantity
Add to cart button	button.cart
Add-to-cart modal close	button.close-modal
Cart link	a[href="/view_cart"]
Cart row	#product-2
Cart product name	td.cart_description h4 a
Cart unit price	td.cart_price p
Cart quantity	td.cart_quantity button
Cart total	td.cart_total p.cart_total_price